End-to-End MLOps Pipeline — Emotion Classification

Group Assignment 3 · MLOps · PGD AI · IIT Jodhpur

Task 2 figures below are real (produced by src/data_prep.py). Remaining <...> blanks (links, V1/V2 metrics, screenshots) come from your Kaggle / W&B / GitHub runs. Every link must be public and live at submission.

1. Team & contributions

Roll No.	Name	Contribution
G25ait2083	<name>	Repo admin & branch protection, W&B project setup
G25ait2046 (me)	<name>	Data prep & normalisation (Task 2), model loading (Task 3), inference script + Dockerfile (Task 6), CI & inference GitHub Actions (Task 7)
<roll>	<name>	Kaggle training runs & W&B logging (Task 4), HF model push (Task 5)
<roll>	<name>	Report, README, experiment comparison & analysis

Every member must have commits in the GitHub history (check Insights → Contributors before submitting).

2. Live links

Component	URL
GitHub repository	https://github.com/g25ait2083-sys/MLOPS_GROUP_ASSIGN3
Kaggle notebook — V1	<public link>
Kaggle notebook — V2	<public link>
Hugging Face model	https://huggingface.co/<username>/distilbert-emotion-mlops-a3
Docker image	https://hub.docker.com/r/<dockerhub>/mlops-a3-emotion
W&B project dashboard	https://wandb.ai/<entity>/mlops-assignment3

3. Git repository setup (Task 1)

- Public repo with README.md, .gitignore, LICENSE.
- main protected — at least 1 PR review required to merge; develop is the integration branch.
- Owner is Admin; all teammates added as Collaborators with Write access.

Screenshot 1 — Collaborators & roles: <paste Settings → Collaborators>

Screenshot 2 — Branch protection on main: <paste Settings → Branches>

4. Data cleaning & normalisation (Task 2)

Dataset: dair-ai/emotion (split config) — 20,000 short English texts labelled with one of six emotions. Splits: 16,000 train / 2,000 validation / 2,000 test. Labels are read directly from the dataset's ClassLabel schema: 0=sadness, 1=joy, 2=love, 3=anger, 4=fear, 5=surprise.

Inspection — class distribution (counts and % per split):

Emotion	Train (n / %)	Validation (n / %)	Test (n / %)
sadness	4666 / 29.2%	550 / 27.5%	581 / 29.1%
joy	5362 / 33.5%	704 / 35.2%	695 / 34.8%
love	1304 / 8.2%	178 / 8.9%	159 / 8.0%
anger	2159 / 13.5%	275 / 13.8%	275 / 13.8%
fear	1937 / 12.1%	212 / 10.6%	224 / 11.2%
surprise	572 / 3.6%	81 / 4.0%	66 / 3.3%
Total	16000	2000	2000

The corpus is clearly class-imbalanced: joy (33.5%) and sadness (29.2%) dominate, while surprise is the rarest at just 3.6% of the training set. The split proportions are consistent across train/validation/test, so the imbalance is inherent to the data rather than a bad split. There were no missing labels.

Cleaning decisions (and why):

- Lowercasing — we fine-tune distilbert-base-uncased, which lowercases internally; normalising on disk keeps stored data faithful to model input.
- Strip URLs and @mentions — they carry no emotional signal and add noise.
- Collapse repeated whitespace — uniform spacing, no semantic change.
- Drop empty rows after cleaning — none occurred here, but the guard is in place.
- Drop exact-duplicate texts — prevents the same sample leaking across splits, which would inflate metrics.
- Punctuation kept by default — '!' and '?' are genuine emotion cues; an opt-in --strip-punctuation flag is available if needed.

Cleaning summary (actual output of src/data_prep.py):

Split	Rows in	Rows out	Dropped: missing	Dropped: duplicates
train	16000	15969	0	31
validation	2000	1998	0	2
test	2000	2000	0	0

Cleaning removed only 33 exact-duplicate rows in total and no rows became empty, confirming the corpus was already fairly clean. The id2label mapping is saved to data/id2label.json (the only data artefact committed to git).

5. Model selection rationale (Task 3)

We chose distilbert-base-uncased. According to its Hugging Face model card, DistilBERT is a distilled version of BERT-base that retains roughly 97% of BERT's language-understanding performance while being ~40% smaller and ~60% faster. At ~66M parameters (~250 MB) it fine-tunes comfortably within Kaggle's free T4 GPU quota, and short emotion tweets fit easily inside a 128-token window, so the speed/size trade-off costs us almost nothing in accuracy. It is uncased — appropriate for casual social-media text where capitalisation is inconsistent — and the model card documents that it was pretrained on the same BookCorpus + English Wikipedia data as BERT via knowledge distillation. The AutoModelForSequenceClassification head is initialised with our six labels and the id2label/label2id maps, so the deployed model emits human-readable class names rather than LABEL_0…LABEL_5. (≈130 words)

6. Experiment comparison (Task 4)

Hyperparameter	Version 1	Version 2
Learning rate	3e-5	5e-5
Train batch size	16	32
Epochs	3	4
Weight decay	0.0	0.01

Metric (held-out test)	Version 1	Version 2
Accuracy	<v1>	<v2>
Weighted F1	<v1>	<v2>
Validation loss	<v1>	<v2>

Which performed better and why: <fill from your W&B runs — name the winner and explain, e.g. V2's higher LR + larger batch converged faster while weight decay curbed overfitting on the dominant joy/sadness classes. Reference the W&B curves.>

Screenshot 3 — W&B dashboard showing both runs: <paste Runs comparison table with accuracy / F1 / loss side by side>

7. Docker design (Task 6)

- Base: python:3.11-slim — small and matches the CI Python version.
- CPU-only PyTorch installed from download.pytorch.org/whl/cpu so the image is a few hundred MB instead of multi-GB CUDA.
- ARG HF_MODEL_NAME with a sensible public default, overridable at build.
- Installs only requirements.txt (inference deps) — no training stack.
- Runs as a non-root user; INPUT_TEXT supplied at run time.

```
docker build --build-arg HF_MODEL_NAME=<username>/distilbert-emotion-mlops-a3 \
             -t mlops-a3-emotion:latest .
docker run --rm -e INPUT_TEXT="I am thrilled about the results!" mlops-a3-
```

```
emotion:latest
docker push <dockerhub>/mlops-a3-emotion:latest
```

Screenshot 4 — successful GitHub Actions inference run (badge or log): <paste the Actions run log from inference.yml showing the printed prediction>

8. Challenges & learnings

- <e.g. wiring Kaggle Secrets + W&B + HF login without leaking tokens>
- <e.g. keeping the Docker image small by using CPU torch>
- Class imbalance: surprise is only 3.6% of the data, so it is the hardest class — watch its per-class F1 in the W&B logs.
- What we'd do differently: <e.g. add class weights / oversampling for surprise, run a W&B Sweep, change one hyperparameter at a time for cleaner attribution>